ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PROJECT REPORT

Steel Surface Defect Recognition with a Convolutional Neural Network



Submitted by:

Arnav Aggarwal (1025150338)

Armaan Malhotra (1025150339)

Tanishq Vashistha (1025150341)

Ishita Sharma (1025150345)

Abhilasha Das (1025150396)

Submitted to:

Dr. Jyoti Maggu

Code and notebooks: https://github.com/ArnavCodez/Ai-for-engineers-EST-project

ABSTRACT

Surface defects on hot-rolled steel strip lower product quality and lead to rejected coils, and checking the strip by eye is slow and tiring. In the Tata Steel scenario of this project, the aim is a convolutional neural network (CNN) that recognises surface defects, including cracks, from images, so that defect counts can later point to faulty equipment.

Earlier work covers manual inspection, hand-designed texture features such as completed local binary patterns, and CNNs with transfer learning on small defect sets. High accuracy on one benchmark split does not show whether a model would survive a real plant or what it looks at. This project therefore concentrates on honest evaluation: a split made before any data exploration, a test set opened once, and extra tests of robustness, explanation and size.

We use the public NEU surface defect database: 1,800 grayscale images of 200 x 200 pixels in six classes of 300 images each. Crazing, a network of fine surface cracks, is our crack class. The split is 1,260 training, 270 validation and 270 test images, with one duplicate pair kept together. We compare three CNNs: a small CNN of 109,958 parameters (M1), the same network with batch normalisation and augmentation (M2), and MobileNetV2 pretrained on ImageNet and fine-tuned (M3). Two batch-normalisation bugs had to be found and fixed on the way.

On the test set, M3 reached 98.52% accuracy (4 errors in 270 images) and a macro F1 of 0.9852. It found all 45 crazing images with no false crack alarm. M1 reached 94.07% although it had tied M3 at 97.04% on validation. Grad-CAM heatmaps showed that M1 often looks at the background. When images were made 0.45 darker, M3 kept 91.85% validation accuracy while M1 fell to 25.56%. Gaussian noise of standard deviation 0.10 broke every model. After conversion to TensorFlow Lite, M3 shrinks from 24.46 MB to 2.51 MB and takes 18.03 ms per image on a CPU.

NEU is clean, lab-style data and a single dataset, so these results do not yet show how the model would perform on a production line.

1. Introduction

1.1 Background and Problem Overview

A hot-rolled steel strip starts as a slab heated to roughly 1100 to 1250 °C. Water sprays remove the oxide layer that forms on the hot surface, rolling mills squeeze the slab thinner in stages, and a coiler winds the finished strip into a coil [1]. Defects can appear at several of these steps. Casting leaves non-metallic inclusions in the steel, weak descaling presses scale into the surface, and debris or a misaligned roll gap scratches it [2].

Surface defects make the product look worse and can lower its fatigue resistance, meaning how well it survives repeated loading [3]. They also lead to customer dissatisfaction and economic loss [4]. Checking by eye is slow, depends on the inspector's experience, and misses defects when the inspector tires [4]. The strip also moves fast: one article on a hot strip mill quotes speeds of up to 15 m/s [5].

This project follows a Tata Steel scenario. The statement asks for a CNN-based image recognition system that identifies surface cracks in steel plates from high-resolution images, to support predictive maintenance and reduce defects in production. We treat it as an image classification task. The model receives a grayscale image of the steel surface and returns one of six defect classes. The prediction can be written as:

$$\textit{predicted class} = f(\textit{image})$$

where f is the trained CNN. The data is the public NEU surface defect database, so the work is not based on Tata Steel data.

Predictive maintenance means fixing equipment before it fails, using early warning signs. If a classifier counts defect types over time, a rising count points at a part of the line. More rolled-in scale suggests checking the descaling system, and more scratches suggest checking the rolls and the roll gap [2]. One defect type can have several causes, and wrong root-cause analysis is common [6], so a rising count is a reason to inspect and not a diagnosis. We have not built a trend dashboard, and NEU has no timestamps to test one. Our model would supply the labels for such a view.

1.2 Literature Review and Research Gap

Steel surface inspection has moved through several approaches. Manual inspection is the oldest, and its limits are described above [4]. Machine vision then replaced the eye with a camera. Line-scan cameras build an image one row of pixels at a time as the strip passes, which suits a fast-moving strip [5]. Classical methods computed hand-designed texture features from these images. Song and Yan, who also published the NEU database used here, proposed a noise-robust method based on completed local binary patterns for hot-rolled strip surface defects [7]. More recent work uses CNNs. One study classifies steel sheet defects with a NasNet-Mobile CNN trained on few samples [3], and another compares object detection algorithms for surface defect detection [4]. Transfer learning, where a network pretrained on a large photo collection is adapted to a new task, is a common way to cope with small defect datasets. MobileNetV2 is a small network built for mobile devices [9], and the Keras guide on transfer learning and fine-tuning describes the practice we follow [10].

Accuracy on a benchmark does not settle three practical questions. First, was the score measured honestly, with no image shared between training and test, and with the test set kept out of every design decision? Second, does the model look at the defect, or at something in the background that happens to go with the class? Third, would it survive a dirty lens, blur or weaker lights, and could it run on a small computer next to the line? The NEU images are clean and balanced, so high accuracy is expected, and it can hide all three problems.

This project addresses these questions on one dataset. We do not propose a new architecture. We compare three CNNs with a strict evaluation protocol, add Grad-CAM heatmaps to see what the models use, test accuracy under damaged images, measure what augmentation is worth, and convert the models to TensorFlow Lite. We also report the mistakes we made on the way, because two of them (both involving batch normalisation) changed the results.

1.3 Objectives

1. Data check: confirm the size, format, class counts and duplicates of the dataset before using it.
2. Split and exploration: split the data first, then study the training images only, to learn what separates the six defects.
3. Pre-processing: build a loading pipeline and a set of augmentations that make physical sense for a steel strip.
4. Model development: train and compare CNNs of our own and a pretrained MobileNetV2.
5. Evaluation: measure accuracy and macro F1 on a test set opened once, and report precision, recall and F1 for cracks (the crazing class) against the other five classes.
6. Interpretation and extras: show where the models look (Grad-CAM), how they behave on damaged images, what augmentation is worth, and how small and fast the model can get.

We did not fix numeric targets for these measures before training. The success criteria we agreed on were macro F1, recall on cracks (a missed crack reaches the customer, while a false alarm only costs a re-check), time per image, and model size.

1.4 Expected Outcomes

We expect a working pipeline from raw images to a trained classifier, a comparison of three CNNs, and per-class results that show which defects get confused. We expect high accuracy, because NEU is a clean benchmark, and we planned tests to show how far that accuracy carries over when images get worse. The result is meant as a recogniser of defect types for study and for a future inspection aid. It is not a finished inspection system, and its use on a real line would need a plant's own labelled images.

2. Methodology

2.1 Dataset Description and Source

We use the NEU surface defect database from Northeastern University, China [7]. It has 1,800 grayscale images of hot-rolled steel strip, 300 for each of six defect classes [7]. We downloaded it from a Kaggle mirror [8] that uses the NEU-DET layout (images in train and validation folders, plus XML box annotations for object detection). We use only each image and its class, which is the start of its file name. We ignore the mirror's own folders (1,440 and 360 images) and the 1,800 XML files, and make our own split.

Table 1 summarises what our data check (notebook 01) found.

Table 1. Dataset summary.

Item	Value
Source	NEU surface defect database, Kaggle mirror kaustubhdikshit/neu-surface-defect-database [8]
Images	1,800 (6 classes x 300)
Classes	crazing, inclusion, patches, pitted surface, rolled-in scale, scratches
Size and format	200 x 200 px, JPEG
Colour mode	stored as RGB, but the three channels are identical in all 1,800 images, so the images are grayscale
Exact duplicates	1 pair: patches_101.jpg and patches_105.jpg (byte-identical, same class)

The six classes look different from each other. Our sources describe crazing as a network of fine surface cracks, inclusion as foreign material in the surface, patches as irregular blotchy areas, pitted surface as small pits, rolled-in scale as oxide pressed into the surface, and scratches as linear marks from abrasion [4][7]. Rolled-in scale and crazing share visual similarities [4]. We found no source that explains the cause of crazing or of patches, so we make no claim about them.

Cracks. The problem statement asks for surface cracks, but NEU has six defect classes. Crazing is the crack-like class. We decided to train one CNN on all six classes, which keeps all the data, and to report precision, recall and F1 for crazing against the other five classes. We call these the crack metrics.

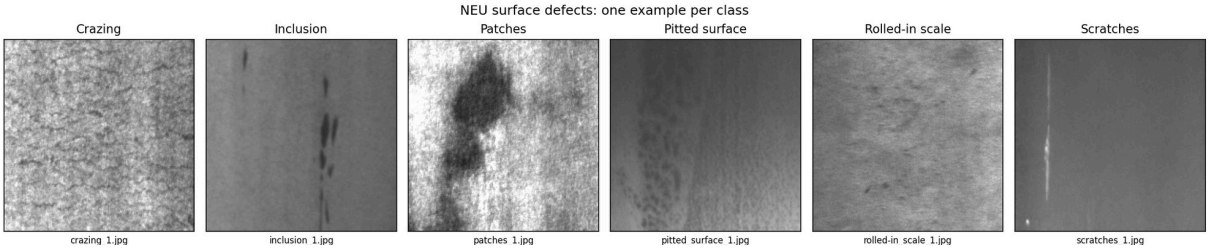


Figure 1. The first image of each class by file name: crazing, inclusion, patches, pitted surface, rolled-in scale and scratches. The defects already look different: dark blobs for patches, a bright line for scratches, small dark marks for inclusion.

Image size and a real line. NEU images are 200 x 200 px, which is small. A line-scan camera produces long, high-resolution images, and a real system would cut such an image into tiles and classify each tile. This is our reading of how NEU relates to a real line. We found no source that says NEU images are crops of line-scan output. We did not build the tiling step.

2.2 Data Preprocessing

Split before exploration. Our original plan did the exploratory data analysis (EDA) first. We moved the split in front of it, so that no design choice depends on the test images. We made a stratified 70/15/15 split with seed 42 (stratified means each class keeps the same share in every part). The split works on groups of identical pixels, so the duplicate pair always lands in the same part. An assertion checks that no picture is in two parts. We ran the notebook a second time and the three split files were byte-identical.

Table 2. Images per class in each part of the split.

Class	Train	Validation	Test
Crazing	210	45	45
Inclusion	210	45	45
Patches	210	45	45
Pitted surface	210	45	45
Rolled-in scale	210	45	45
Scratches	210	45	45
Total	1,260	270	270

Exploratory data analysis. Six plots on the 1,260 training images (class balance, a grid of samples, mean image per class, pixel histograms, brightness and contrast boxplots, PCA of raw pixels) gave these findings:

- The classes are balanced, so accuracy is a fair first metric.
- Brightness alone cannot separate the classes. Median brightness runs from 90.36 (scratches) to 186.05 (pitted surface), but the ranges overlap.
- Patches stand out by contrast: median pixel standard deviation 57.75, against 11.18 to 28.55 for the other classes.
- Patches and pitted surface have a spike at pixel value 255, so some images are over-exposed.
- The lighting is uneven. The mean images of crazing, inclusion, pitted surface and scratches get darker towards the right edge, an effect of the camera and lights that a model should not learn.
- The mean image of scratches shows both vertical and horizontal lines, so scratches run in both directions.
- In raw pixel space the first principal component holds 70.63% of the variance and is simply overall brightness (correlation 1.0). The second holds 8.28%. Crazing, inclusion and rolled-in scale overlap in the middle.

The last point shows why a CNN is needed. The largest difference between images is brightness, which says nothing about the defect. Overlap in two components does not prove the classes are inseparable, only that raw pixels are a poor representation.

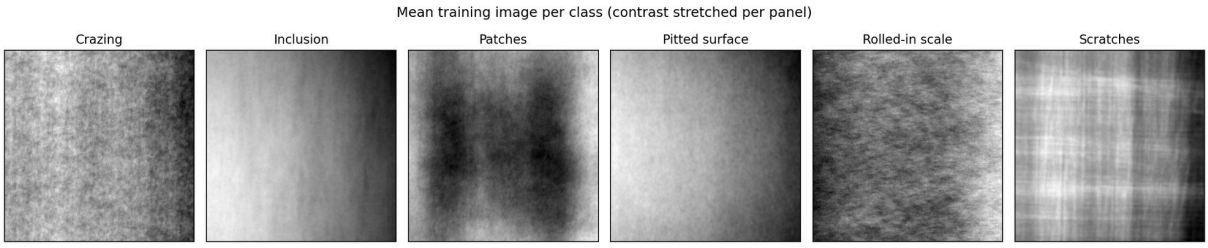


Figure 2. Mean training image per class. Four of the six means get darker towards the right edge (uneven lighting), the scratches mean shows lines in both directions, and the patches mean shows dark blobs near the centre.

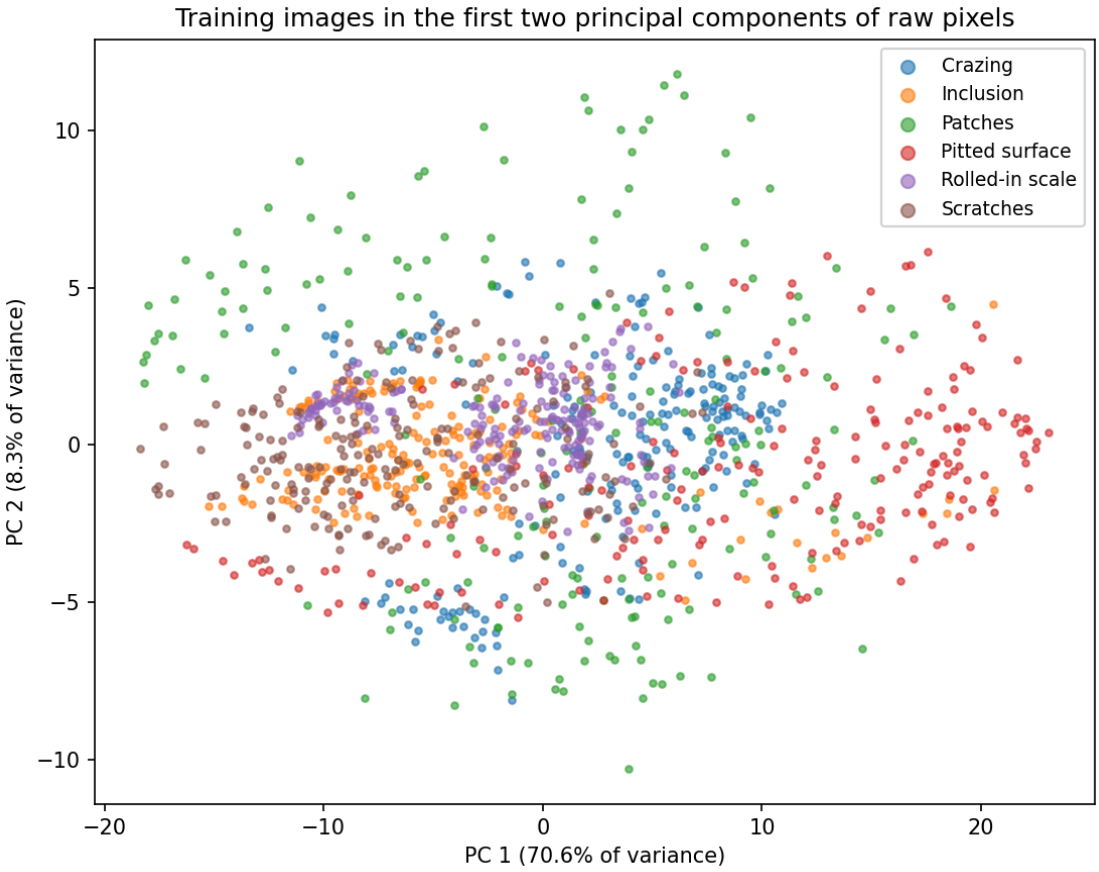


Figure 3. PCA of the training images shrunk to 50 x 50 px. The first component is overall brightness. The classes overlap, so raw pixels do not separate them.

Loading pipeline. A `tf.data` pipeline reads each file, decodes it as one grayscale channel, resizes it and groups images into batches of 32. Our own CNNs take 200 x 200 x 1 images with pixels scaled to 0 to 1. MobileNetV2 takes 224 x 224 x 3 images, with the grayscale channel repeated three times and its own `preprocess_input` scaling to -1 to 1. The scaling matters: feeding a pretrained network the wrong range does not raise an error, it just trains badly. We checked the batches (32 x 200 x 200 x 1 with values 0.0 to 1.0, and 32 x 224 x 224 x 3 with values -0.997 to 1.0), and both first batches contained all six classes.

Augmentation. With 210 training images per class, a network can memorise its training set. Augmentation shows it a slightly different version of each image in every epoch. We chose five layers that make sense for a camera looking at a steel strip: horizontal and vertical flips, rotation up to 5% of a turn either way (about 18 degrees), zoom up to 10%, contrast 20% and brightness 20%. The flips

follow from EDA (uneven lighting, scratches in both directions). We avoid 90 degree turns, because scratches and rolled-in scale follow the rolling direction. Colour jitter is meaningless for grayscale images. Rotation corners are filled by reflecting the image, so there are no black triangles that never occur on a real strip. The layers sit inside the model, and Keras runs them only in training, so validation and test images are never augmented.

Before training we ticked a leakage checklist: the split was made before any augmentation, validation and test images are never augmented, no picture is in two parts, and the test set had not been used for any decision.

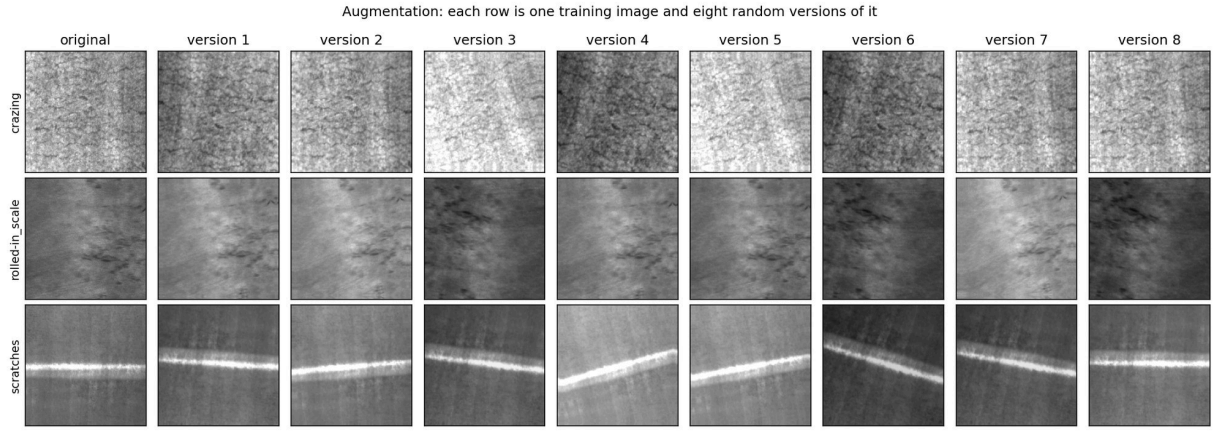


Figure 4. One training image each of crazing, rolled-in scale and scratches, with eight random augmented versions of each. The augmented images still look like real steel.

2.3 Flow Chart

Figure 5 shows the full workflow in four blocks: data, visualisation and pre-processing, modelling, and evaluation with extras. The dotted arrow shows how the EDA findings led to the choice of flips.

Figure 5. Project flow chart, from the data check to the extras. The training settings shown are the final ones from Section 2.4 and Section 3.1.

2.4 Model Architectures

All models are trained with Adam and sparse categorical cross-entropy, which is small when the model gives high probability to the true class.

M1, baseline CNN (no augmentation). Three convolution blocks. Each block has a 3 x 3 convolution with ReLU, which slides small filters over the image and responds to edges and textures, followed by 2 x 2 max pooling, which halves the image size and keeps the strongest response. The filters grow from 32 to 64 to 128 as the image shrinks, so later layers see larger patterns. Global average pooling turns each of the 128 feature maps into one number. A dense layer of 128 units with ReLU, dropout of 0.5 and a 6-way softmax layer give the class probabilities. M1 has 109,958 parameters and a 1.38 MB file.

M2, improved CNN. The same network with batch normalisation after each convolution and the augmentation layers from Section 2.2. Batch normalisation rescales the activations of each batch, which steadies training. At prediction time it uses running averages collected during training. Keras updates these with a momentum of 0.99 by default, and for M2 we had to change this to 0.9 (see Section 3.5). M2 has 110,630 parameters and a 1.41 MB file.

M3, MobileNetV2 transfer learning. MobileNetV2 is a small network built for mobile devices, using depthwise convolutions and inverted residual blocks [9]. We load weights learned on ImageNet, add a head of global average pooling, dropout of 0.3 and a 6-way dense layer, and train in two stages:

- Stage A: the base is frozen and only the head learns, with learning rate 1e-3, for up to 20 epochs.
- Stage B (fine-tuning): the base is unfrozen from layer 100 (its last six blocks, about a third of its layers) and training continues at learning rate 1e-4, for up to 40 epochs. Every batch-normalisation layer of the base stays frozen, and the base is called with `training=False`.

The frozen batch-normalisation layers were the result of a bug, described in Section 3.5. M3 has 2,265,670 parameters and a 24.46 MB file.

3. Experiments and Results

3.1 Experimental Setup

Table 3. Experimental setup.

Parameter	Configuration
Task	6-class image classification, plus crack (crazing) versus rest metrics
Framework	TensorFlow 2.20.0 with Keras, Python 3.13.15, NumPy 2.1.3, pandas 2.3.3
Hardware	Kaggle notebooks with a Tesla T4 GPU
Random seed	42
Data	1,260 training, 270 validation, 270 test images
Batch size	32
Optimiser, loss	Adam, sparse categorical cross-entropy
M1 and M2 training	learning rate 1e-3, up to 100 epochs, early stopping on validation loss with patience 12, learning rate halved after 4 epochs without improvement
M3 training	stage A lr 1e-3 (up to 20 epochs, patience 5), stage B lr 1e-4 (up to 40 epochs, patience 8), batch normalisation frozen
Checkpoint	weights of the epoch with the lowest validation loss

The notebooks run on Kaggle, with the NEU mirror attached, and the exact library versions of that run are saved in the repository. The seven notebooks are: data check, split and EDA, pipeline and augmentation, own CNNs, transfer learning, evaluation, and extras. We raised the early-stopping patience of our own CNNs from 8 to 12, because the validation curve of M2 is jumpy and training stopped before the learning-rate cuts could calm it.

3.2 Comparison Method

We compare three models on identical data and the same split: M1, M2 and M3. M1 is the simplest CNN we can explain layer by layer. M2 shows what batch normalisation and augmentation add. M3 shows what a pretrained network adds. Note that M2 differs from M1 in two ways at once, so a difference between them cannot be credited to one change alone. We did not include a non-neural baseline. The comparison is among CNNs only.

3.3 Evaluation Protocol

The protocol was fixed before training:

1. All model choices (architecture, learning rates, epochs, which models are finalists) use the validation set only.
2. We chose two finalists on validation: M3, which had the lowest validation loss, and M1, the best of our own CNNs and about 18 times smaller. We logged this decision first.
3. The test set (270 images, 45 per class) was opened once, at 04:38:53 UTC on 18 October 2026 (10:08:53 IST), about two minutes after the finalists were logged. All three models were scored in that single pass so the comparison table is complete, but only the finalists count as our reported result. Nothing was changed afterwards.

4. The extra experiments that damage images or retrain models (robustness, ablation) use the validation set, so the test set stays a one-time measurement.

Accuracy is the share of images classified correctly. *Precision* for a class is the share of images predicted as that class that truly belong to it. *Recall* is the share of true images of that class that the model found. *F1* is the harmonic mean of the two:

$$F1 = 2 \times precision \times recall / (precision + recall)$$

Macro averages are computed per class and then averaged, so every class counts the same. The crack metrics are precision, recall and F1 for crazing against the other five classes. Recall is the one we care about most, because a missed crack reaches the customer. CPU time per image is the mean of 100 single-image calls on the CPU after 10 warm-up calls. Model size is the size of the saved file.

One caution applies. With 270 images in a set, one image is 0.37 percentage points. Validation was also used to compare models, to stop training and to settle the fine-tuning recipe (three tries, Section 3.5), so validation scores are a little optimistic. Each model was trained once, so we have no standard deviations.

3.4 Quantitative Results

Validation results

Table 4. Validation results (270 images). Epochs shows epochs trained and, in brackets, the best epoch. For M3 the 29 epochs are 15 in stage A and 14 in stage B.

Model	Val. accuracy	Val. loss	Parameters	File size	Epochs (best)	Train time
M1 baseline CNN	97.04%	0.1056	109,958	1.38 MB	100 (100)	182.1 s
M2 + batch norm + augmentation	95.93%	0.1310	110,630	1.41 MB	41 (29)	158.6 s
M3 MobileNetV2	97.04%	0.0832	2,265,670	24.46 MB	29 (21)	121.2 s

M1 and M3 tie on validation accuracy. M3 has the lowest validation loss, 0.0832 against 0.1056 for M1 and 0.1310 for M2, which is why we chose it. M1 is not overfitting: its training accuracy, measured with dropout on, was below its validation accuracy in 91 of 100 epochs, and it was still improving slowly at the 100-epoch limit. M2 scored 95.93%, three validation images fewer than M1. On a small, clean dataset, batch normalisation and augmentation did not beat a tiny CNN that already has dropout and global average pooling. M2's validation curve is jumpy: after epoch 10 its accuracy still moves between 72.6% and 96.3%. With frozen ImageNet features alone (stage A), M3 peaked at 81.11%, so fine-tuning matters on steel textures.

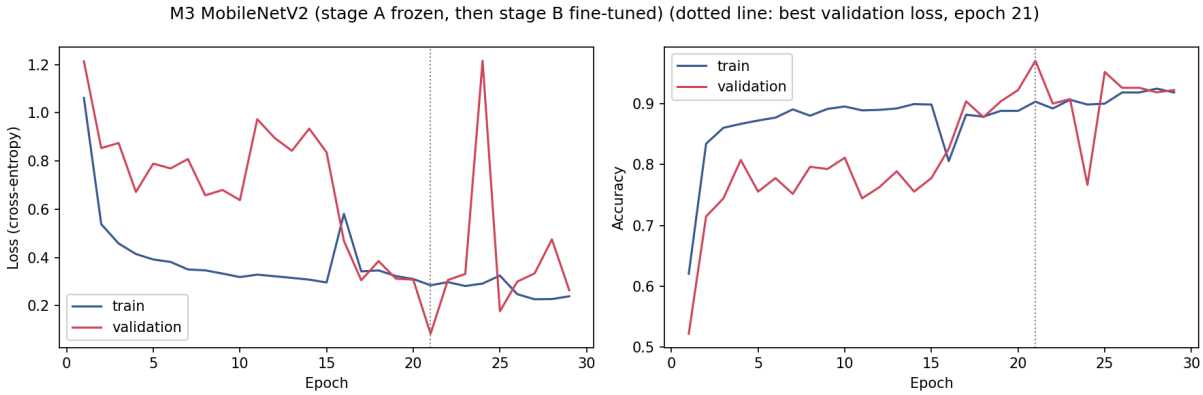


Figure 6. Training curves of M3. Stage A covers epochs 1 to 15 and stage B epochs 16 to 29. The dotted line marks the epoch with the lowest validation loss (21). The validation curve is jumpy, as expected with 270 images. Training accuracy is measured with dropout on, so it can sit below validation accuracy.

Test results

Table 5. Test results (270 images, opened once). Crack metrics are for the crazing class against the other five. M2 was not a finalist.

Model	Accuracy	Macro F1	Crack precision	Crack recall	Crack F1	Errors	CPU ms per image	File size
M3 MobileNetV2 (finalist, final model)	98.52%	0.9852	1.0000	1.0000	1.0000	4	223.96	24.46 MB
M1 baseline CNN (finalist)	94.07%	0.9400	0.9783	1.0000	0.9890	16	21.57	1.38 MB
M2 improved CNN (not a finalist)	96.67%	0.9665	0.9375	1.0000	0.9677	9	29.31	1.41 MB

M3 is our final model: 98.52% accuracy with 4 errors in 270 images, a macro precision of 0.9860 and a macro recall of 0.9852. Every model found all 45 crazing test images, so crack recall is 100% for all three. M3 raised no false crack alarm. M1 called one patches image crazing (crack precision 0.9783).

M1 dropped from 97.04% on validation to 94.07% on test, while M2 rose from 95.93% to 96.67%. This is the opposite of the validation ranking. Validation could not separate models this close. We kept the finalists we had chosen before opening the test set and did not switch to M2. M3 ranked above M1 on test accuracy, as its lower validation loss had hinted, but the gap between M1 and M2 on test should not be read as a clear verdict, because it comes from a single run of each.

Speed and size differ widely. On Kaggle's CPU, one image takes 21.57 ms with M1 and 223.96 ms with M3, about 10 times slower, and M3's file is about 18 times larger. M3 is the more accurate choice, while M1 suits a very small computer. These times were measured on Kaggle's CPU, and a laptop or an edge board will differ.

Robustness

A steel plant is hot, steamy and dusty, and lights age. We damaged the validation images in four ways, each at three strengths, and measured accuracy again. We also tested M2, because it is the one CNN trained with brightness and contrast augmentation. Table 6 gives validation accuracy in percent at the three strengths, from mildest to strongest (clean accuracy: M1 97.04, M2 95.93, M3 97.04).

Table 6. Validation accuracy (%) under damage, at three strengths from mild to strong.

Damage (strengths)	M1	M2	M3
Gaussian noise (std 0.02 / 0.05 / 0.10 on a 0 to 1 scale)	58.2 / 29.3 / 24.4	46.3 / 31.1 / 25.2	96.3 / 82.6 / 48.1
Gaussian blur (kernel 3 / 5 / 9 px)	38.5 / 32.6 / 25.6	72.2 / 47.4 / 38.5	95.9 / 92.6 / 84.8
Darker (by 0.15 / 0.30 / 0.45)	65.9 / 31.9 / 25.6	89.6 / 52.6 / 42.6	96.7 / 96.7 / 91.9
Low contrast (x 0.8 / 0.6 / 0.4)	80.0 / 38.1 / 24.8	97.8 / 94.1 / 52.6	95.9 / 87.0 / 65.2

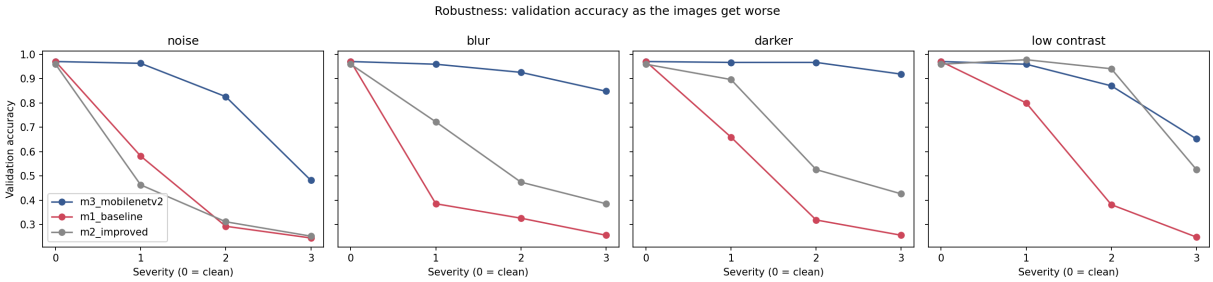


Figure 7. Validation accuracy of M3, M1 and M2 as the images get worse (severity 0 is the clean image). Run on validation images only.

MobileNetV2 holds up best: 92.6% with a 5 px blur and 91.9% with the image 0.45 darker. M1 collapses quickly: 65.9% at the first darkening step, 38.5% with a 3 px blur, and 58.2% with slight noise. M2 is far more stable than M1 under lighting changes (89.6% against 65.9% one step darker, and 94.1% against 38.1% at contrast x 0.6), but not under noise. Strong noise (std 0.10) hurts every model, and M3 drops to 48.1%. None of our augmentations adds noise.

TensorFlow Lite

TensorFlow Lite turns a Keras model into a compact file for small boards. We used dynamic-range quantisation, which stores the weights as 8-bit integers instead of 32-bit floats. We converted an inference copy of each model without the augmentation block, because the first attempt on MobileNetV2 failed: the augmentation layers contain random operations that TensorFlow Lite cannot convert, and they do nothing at prediction time anyway.

Table 7. TensorFlow Lite conversion, checked on the validation set.

Model	Keras size	TFLite size	Keras val. accuracy	TFLite val. accuracy	Same answer as Keras	TFLite CPU ms per image
M3 MobileNetV2	24.46 MB	2.51 MB	97.04%	97.41%	99.63%	18.03
M1 baseline CNN	1.38 MB	0.12 MB	97.04%	97.04%	100%	10.14

M3 shrinks about ten times to 2.51 MB and takes 18.03 ms per image on Kaggle's CPU, against 223.96 ms for the Keras model in the test notebook. M1 shrinks to 0.12 MB and takes 10.14 ms, against 21.57 ms for Keras. The two timings were taken in different notebooks on the same kind of CPU, so they indicate the size of the gain and not an exact ratio. M3's TensorFlow Lite version gave a different answer from the Keras model on one of 270 images and happened to get it right. This is quantisation noise, not an improvement. We have not run either model on an actual edge board.

3.5 Qualitative Results

Where the models look (Grad-CAM). Grad-CAM asks how much the score of the predicted class would change if a part of the last convolution layer's output changed [11]. Regions with a large effect light up on the heatmap. We wrote it by hand with `tf.GradientTape`. For M1 we used the last ReLU before pooling, and for M3 the output of the pretrained base (7 x 7 cells, 1,280 channels). We drew two validation images per class for each finalist.

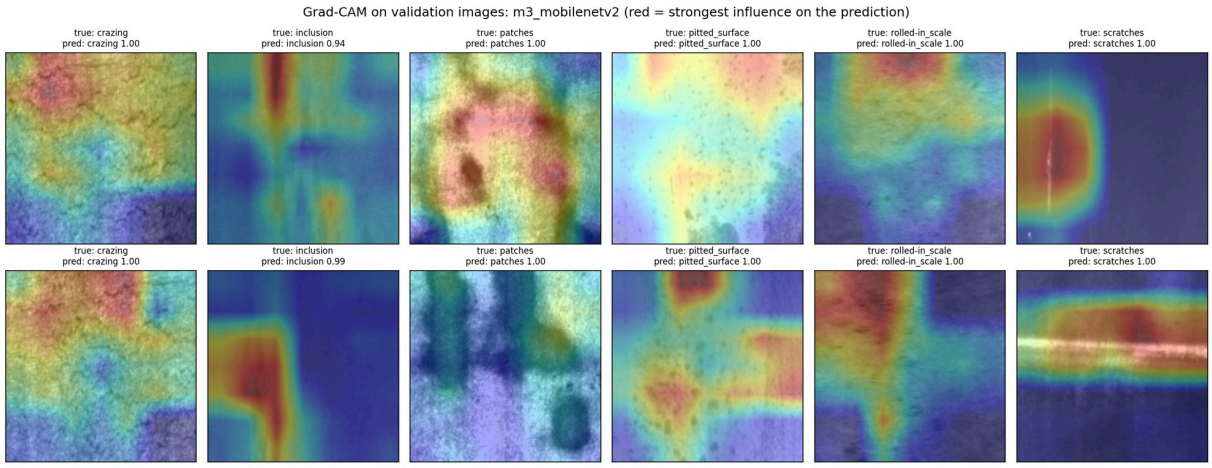


Figure 8. Grad-CAM heatmaps of M3 on two validation images per class. The bright regions sit on the defects: the scratch line, the inclusion streak and the dark patches.

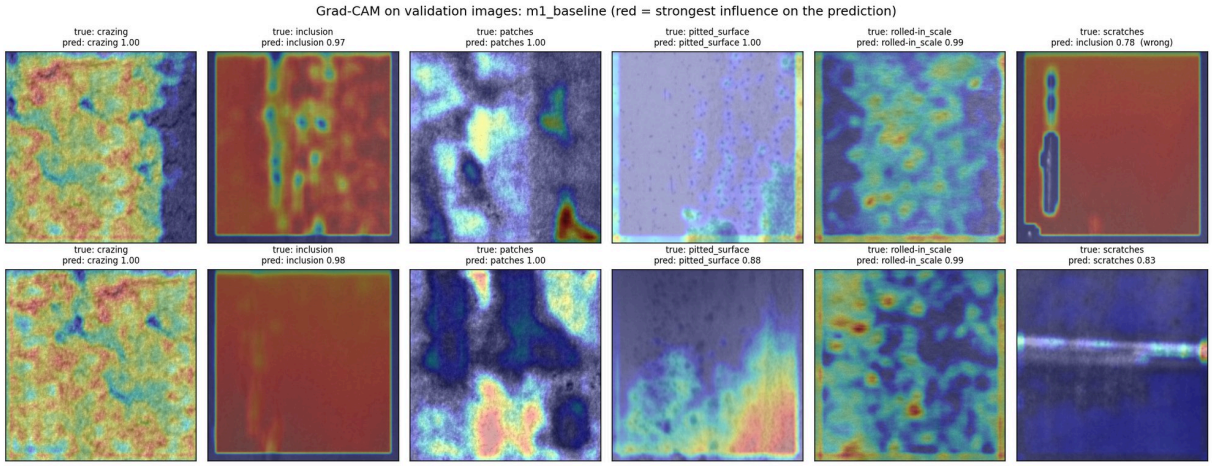


Figure 9. Grad-CAM heatmaps of M1 on the same images. For inclusion, the flat steel around the defect lights up, and on a missed scratch the background is red and the scratch is not.

M3's heatmaps sit on the defects. M1's often sit on the background. EDA showed that inclusion images are among the flattest, so a likely reading is that M1 learned "flat background means inclusion", a shortcut based on brightness and texture. This fits the numbers: M1 collapses when the images are darkened or blurred, and most of its test errors are in pitted surface. This is our interpretation of the heatmaps. We did not test it with a separate experiment.

What the models get wrong. Figure 10 shows the confusion matrix of M3. Its four mistakes are three faint scratches called inclusion and one rolled-in scale image called pitted surface. A thin line along the rolling direction looks much like a long inclusion. M1 made 16 mistakes, and 10 of them are pitted-

surface images (pitted surface recall 77.78%), called inclusion (6), scratches (3) or rolled-in scale (1). Nine of those ten were predicted with a confidence below 0.7. Scratches recall for M1 is 88.89%, with 4 of its errors called inclusion. Our sources suggested that rolled-in scale and crazing would be confused [4], but neither finalist confused them on the test set.

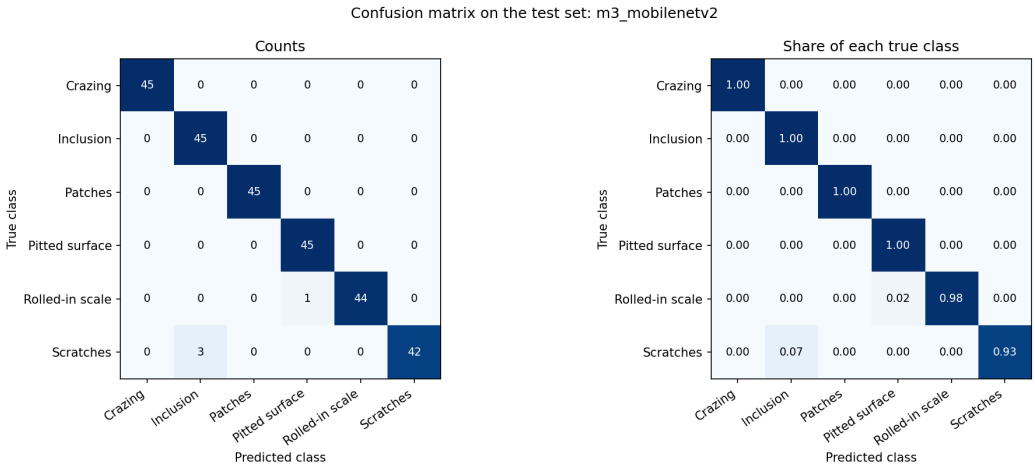


Figure 10. Confusion matrix of M3 on the test set. Rows are true classes and columns are predicted classes. Counts are on the left and row-normalised shares on the right.

Bugs we found and fixed. Two bugs involved batch normalisation, and both showed up only through validation results.

The first was in M2. In our first run, M2 scored exactly 16.67% validation accuracy in every epoch, which is one in six, the signature of a model that outputs one class for every image. Its training accuracy rose from 49.5% to 88.5%, so only the validation side was broken. We opened the saved model and read the batch-normalisation statistics. Batch normalisation uses each batch's own statistics in training but running averages at prediction time, and Keras updates those averages with momentum 0.99, only 1% per step. With 40 batches per epoch they lagged far behind: in the first layer the running variance was about 0.67 in every channel, while the real variance of those activations is about 0.01. Every validation image was normalised wrongly. Setting the momentum to 0.9 let the averages catch up within an epoch. M2 then reached 95.19% in the check run and 95.93% in the final run.

The second was in the fine-tuning of M3. Our first try unfroze the top 30 layers at learning rate 1e-5 and reached 88.89%. A second try, from layer 100 at 1e-4, did worse (81.11% at its best epoch), and validation accuracy jumped between 50% and 80%. In both, training accuracy fell at the first fine-tuning epoch (from 0.90 to 0.51 in the first try), although such a small learning rate barely changes the weights. The cause was again batch normalisation: once unfrozen, the base's layers normalised with the statistics of each steel batch instead of the ImageNet statistics, even though the base is called with `training=False`. The reliable fix in Keras 3 was to keep every batch-normalisation layer frozen. The third try (from layer 100, 1e-4, batch normalisation frozen) reached 97.41% in the check run and 97.04% in the final run. Fine-tuning is not exactly repeatable on a GPU, and only the final run counts. The failed runs are kept in the repository. Because we tried three recipes on the validation set, M3's validation score has had a little more tuning than M1's.

3.6 Ablation

The ablation tests what augmentation is worth. We retrained the M2 network three times with everything identical except the augmentation, on the validation set only.

Table 8. Augmentation ablation (M2 network, validation set).

Augmentation	Val. accuracy	Val. loss	Epochs trained
None	100.00%	0.0147	91
No rotation (flips, zoom, contrast, brightness)	99.26%	0.0558	45
Full (as in M2)	95.56%	0.1677	38

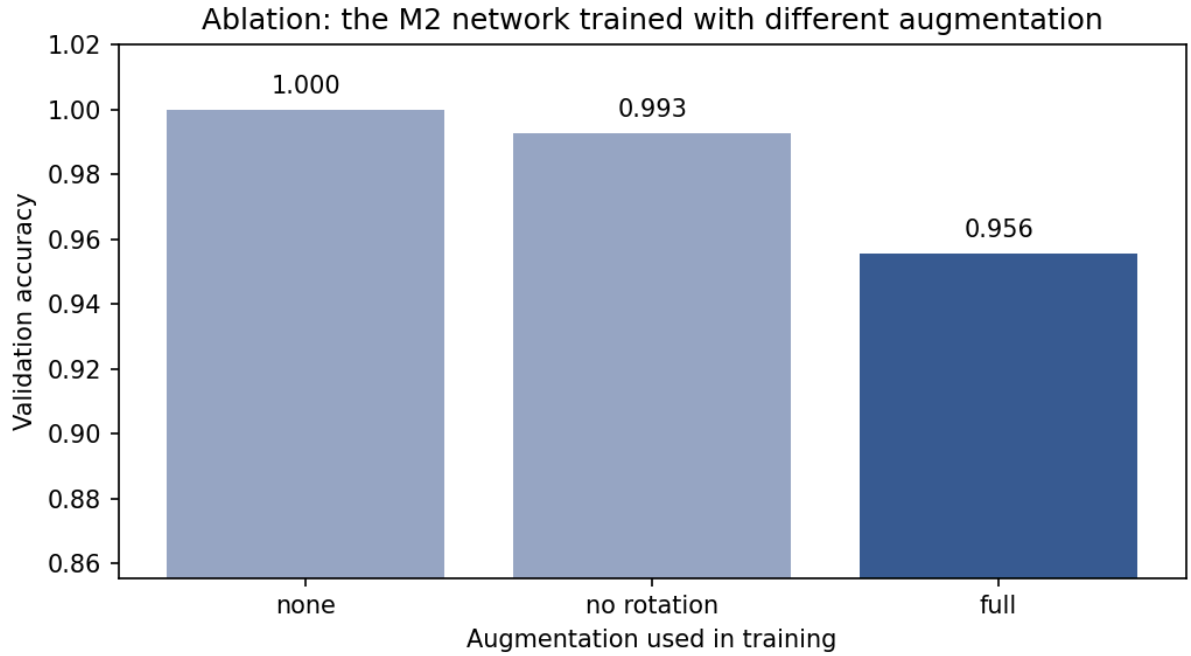


Figure 11. Validation accuracy and loss of the M2 network with no augmentation, no rotation and the full set.

On clean NEU images, augmentation costs accuracy, and rotation costs the most. This fits what we expected: scratches and rolled-in scale follow the rolling direction, so a rotated image is less realistic. An earlier run of the same notebook gave the same order (100.00%, 98.89%, 94.07%), although training on the GPU is not exactly repeatable and the exact numbers differ. The full-augmentation retrain (95.56%) also differs slightly from M2 in Table 4 (95.93%).

The other side of the trade-off is Table 6. Augmentation is what keeps M2 working when the lighting changes, which on a real line it will. The best clean score (100% on validation) came from the network with no augmentation. We did not test it on the test set, because the test set had already been used. A 100% score on a validation set that we used for several decisions should not be taken at face value.

3.7 Discussion

The results support four conclusions.

First, a pretrained network helps with small data, but only when fine-tuned with care. Frozen ImageNet features reached 81.11% on validation, and fine-tuning with batch normalisation frozen

brought M3 to 97.04% on validation and 98.52% on test. Our own small CNN reached 94.07% on test with 18 times fewer parameters. Whether to pay for the larger model depends on the device: M3 takes 223.96 ms per image on Kaggle's CPU before quantisation, and 18.03 ms after.

Second, validation could not separate M1 from M3 (both 97.04%), and it ranked M1 above M2 where the test set ranked M2 above M1. The explanation is the size of the sets: 270 images, with one image worth 0.37 points. We chose the finalists on validation loss and left them unchanged after seeing the test. This is the protocol working as designed, even though it produced an awkward result for M1. It also means we cannot claim a reliable ranking of M1 and M2 from this data.

Third, a high accuracy does not show that a model has learned the defect. M1 and M3 both scored well on clean validation images (97.04%), yet Grad-CAM and the robustness test show that M1 relies on the background and breaks under small changes in brightness, blur or noise. For a plant, where lighting changes and lenses get dirty, this difference matters more than the 4 points of test accuracy between them.

Fourth, crack recall was 100% for all three models on the 45 crazing test images, and M3 had no false alarm. This is a good result, but it rests on 45 images from one lab-style dataset. It does not mean that a crack would never be missed on a line.

Limitations. NEU is clean, balanced, lab-style data, which is why accuracy is high. Its images are 200 x 200 crops of what a line camera sees, and we did not build the tiling step for long high-resolution images. We use one dataset and one split. Each model was trained once, so we have no spread across seeds. Noise breaks every model at standard deviation 0.10. Timings come from Kaggle's CPU and not from an edge device, and we did not build a trend view over time. A real plant would need its own labelled images.

4. Conclusion

This project built and evaluated a CNN-based recogniser of six steel surface defects, with crazing as the crack class. We used the public NEU database in a Tata Steel scenario, and compared three CNNs: a small CNN of our own (M1), the same CNN with batch normalisation and augmentation (M2), and MobileNetV2 with fine-tuning (M3).

On a test set opened once, M3 reached 98.52% accuracy and a macro F1 of 0.9852, with 4 errors in 270 images. It found all 45 crazing images with no false crack alarm. M1 reached 94.07% and M2 96.67%. M1 and M3 had tied at 97.04% on validation, and the test set did not agree with validation about M1 and M2. We kept the finalists we had chosen in advance.

The extra experiments gave a more cautious picture than the accuracy alone. Grad-CAM showed that M3 looks at the defect, while M1 often looks at the background. On damaged validation images, M3 degraded gradually under blur, darkening and low contrast, while M1 fell quickly. Noise at standard deviation 0.10 broke every model. Augmentation reduced clean accuracy, most of all with rotation, and gave stability under lighting changes. TensorFlow Lite cut M3 from 24.46 MB to 2.51 MB and its CPU time to 18.03 ms per image.

We also found and fixed two batch-normalisation bugs, which would have made two of our models look much worse than they are. Our results come from clean benchmark data. They show that the method works on NEU, and not yet that it works on a production line.

5. Future Work Plan

1. Test on more data. Try other steel defect datasets and, if possible, images from a real line, to find out whether the results carry over. A plant would need its own labelled images.
2. Build the tiling step. Cut long high-resolution line-camera images into tiles like the NEU crops and classify each tile, which is how a real system would use the model.
3. Add noise to augmentation. No augmentation we used adds noise, and noise broke every model at standard deviation 0.10.
4. Repeat the runs. Train each model several times and report the mean and spread, since GPU training is not exactly repeatable and one image is 0.37 points on our 270-image sets.
5. Use a larger test set for cracks. The 45 crazing test images are too few to show that crack recall is reliably high.
6. Run on an edge device. Measure the TensorFlow Lite models on a small board next to the line, and try full integer quantisation.
7. Build the trend view. Count defect types over time and test whether a rising count points to the right equipment. This needs timestamped data, which NEU does not have.
8. Show the model in use. A simple demo app that takes an image and shows the class and a Grad-CAM overlay.

References

[1] ispatguru, "Rolling of steel in hot strip mill." Available at: https://www.ispatguru.com/rolling-of-steel-in-hot-strip-mill/

[2] Steel Technology, "Rolling Mill Defects, Process Control, and Quality Improvement." Available at: https://www.steel-technology.com/articles/rolling-defects-process-control-steel-quality

[3] "Classification of Surface Defects in Steel Sheets Using Developed NasNet-Mobile CNN and Few Samples," Revue d'Intelligence Artificielle, vol. 38, no. 2. Available at: https://www.iieta.org/journals/ria/paper/10.18280/ria.380232

[4] A. Maity and T. Ghosh, "Comparative Analysis of Object Detection Algorithms for Surface Defect Detection," arXiv:2510.21811, 2025.

[5] Vision Systems Design, "Linescan cameras inspect steel." Available at: https://www.vision-systems.com/factory/article/16736748/linescan-cameras-inspect-steel

[6] ABM Proceedings, "Hot rolling surface quality issues" (abstract only). Available at: https://abmproceedings.com.br/en/article/download-pdf/hot-rolling-surface-quality-issues-4

[7] K. Song and Y. Yan, "A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects," Applied Surface Science, vol. 285, pp. 858 to 864, 2013. The class list and image counts were taken from the NEU entry in awesome-industrial-datasets: https://github.com/jonathanwvd/awesome-industrial-datasets/blob/master/markdown/neu_surface_defect_dataset.md

[8] NEU Surface Defect Database (Kaggle mirror, NEU-DET layout). Available at: https://www.kaggle.com/datasets/kaustubhdikshit/neu-surface-defect-database

[9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510 to 4520.

[10] Keras, "Transfer learning and fine-tuning" (guide). Available at: https://keras.io/guides/transfer_learning/

[11] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," in Proc. IEEE International Conference on Computer Vision (ICCV), 2017.

[12] AI for Engineers EST Project, project repository (notebooks, results, report). Available at: https://github.com/ArnavCodez/Ai-for-engineers-EST-project